



Trackvance Core

Especificación técnica v1.1

IMPLEMENTACIÓN 0.3.0 · EVOLUCIÓN CONEXIONES

19 de septiembre de 2026 · Estado local verificado

Arquitectura local y evolución a producto

Monolito modular · Persistencia local · Reglas portables · Evidencia verificable

Documento oficial de referencia. Describe el estado real de Trackvance Core y distingue las capacidades implementadas de los contratos preparados y la infraestructura objetivo.

Trackvance Colombia SAS

Contenido

1. Alcance, estados e invariantes	3
2. Arquitectura lógica estable	4
3. Despliegue local de referencia	5
4. Código, stack y dependencias	6
5. Fuentes y lectores de datasets	7
5A. Módulo Conexiones	8
6. Carga, esquema y versionado	9
7. Profiling, identificadores y transforms	10
8. Reglas portables y Data Intake	11
9. ReconOps y normalización declarada	12
10. Sentinel y métricas históricas	13
11. Excepciones con validación técnica	14
12. Centro de Control y navegación	15
13. StorageProvider, artefactos y linaje	16
14. Manifest schema 2 y auditoría	17
15. Reportes Excel de negocio	18
16. Ejecución, planificación y recuperación	19
17. Modelo persistente y migraciones	20
18. API y errores	21
19. Contratos HTTP y permisos	23
20. Configuración, operación y respaldo	24
21. Arquitectura final de producto	25
22. Calidad, pruebas y aceptación	26
23. Decisiones vigentes y control documental	27

1. Alcance, estados e invariantes

Trackvance Core es una plataforma local de confiabilidad de datos. Integra Data Intake, ReconOps y Sentinel con Datasets, Excepciones, Centro de Control, Auditoría e Identidad/RBAC. El usuario carga o versiona datos, publica una configuración inmutable, ejecuta controles y conserva evidencia verificable del resultado.

Esta especificación describe la entrega 0.3.0 y su evolución con Conexiones del 19 de septiembre de 2026. El documento y la versión del software tienen ciclos distintos: se conserva el nombre v1.1 solicitado y se identifica esta revisión de implementación. Los resultados de pruebas y los límites pendientes aparecen en la sección de aceptación.

Estado	Significado normativo
IMPLEMENTADO	Existe en el código local y en el flujo operativo, con cobertura automatizada o evidencia ejecutada.
PREPARADO	Existe un puerto, contrato versionado o rechazo controlado. El adaptador productivo todavía requiere implementación y validación.
OBJETIVO	Forma parte de la arquitectura final prevista; no es una dependencia ni capacidad operativa de la instalación local.

Invariantes compartidos

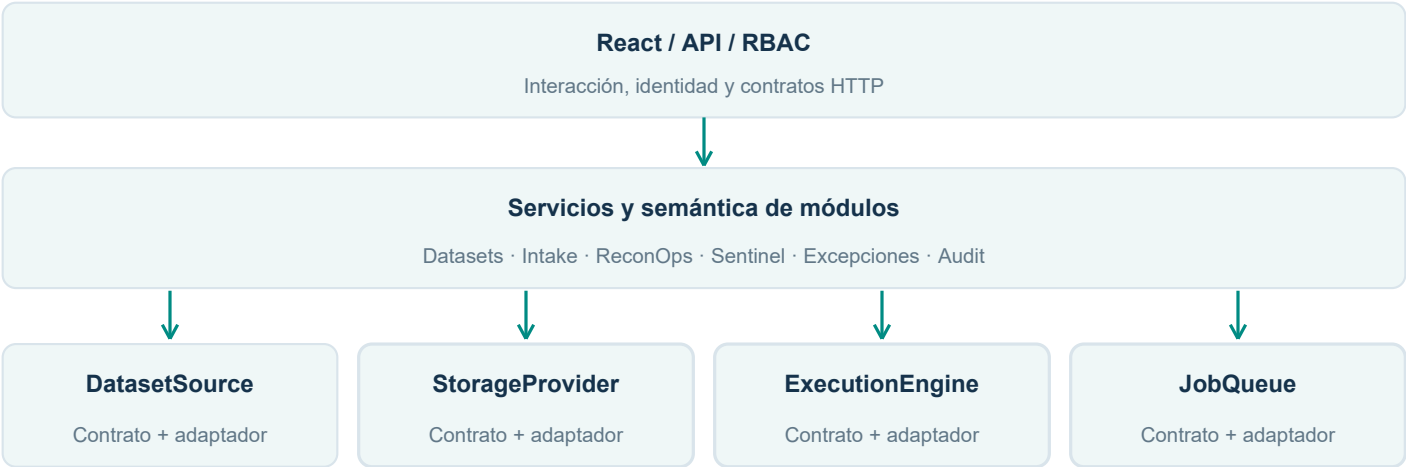
- La arquitectura lógica es un monolito modular. API y workers pueden desplegarse como procesos separados del mismo producto; no son microservicios de negocio.
- PostgreSQL guarda metadata y relaciones. Los archivos recibidos, Parquet canónicos, resultados, manifiestos y exports se guardan fuera de PostgreSQL.
- DatasetVersion, configuración publicada y Run conservan identidades estables. Una modificación funcional crea una versión o ejecución nueva.
- SUCCESS significa que terminó el procesamiento. REJECTED, WITH_FINDINGS y ALERT son decisiones de negocio y no errores técnicos.
- Profiling observa los valores recibidos sin trim, case o normalización Unicode funcional implícita. Las transformaciones de negocio deben estar declaradas.
- Toda modificación del esquema persistente se realiza mediante una migración Alembic nueva. No se editan migraciones ya aplicadas.
- La autorización por rol, permiso y organización se verifica en backend; ocultar un botón no sustituye la autorización.
- El prototipo funciona localmente con imágenes preparadas. No necesita servicios cloud para operar.

Qué no se certifica como implementado

PySpark operativo, Redis/Celery, conectores distintos de PostgreSQL/SQL Server, object storage remoto, OAuth2/OIDC, Kubernetes, Terraform, Helm, observabilidad distribuida, masking por sensibilidad y escalamiento de producción pertenecen a los estados PREPARADO u OBJETIVO. Sus contratos y plan de evolución se detallan más adelante.

2. Arquitectura lógica estable

La separación principal es entre adquisición de datos, almacenamiento interno y ejecución. Estos conceptos tienen responsabilidades y credenciales distintas. Leer un archivo de un bucket del cliente no implica almacenar los artefactos internos en ese mismo bucket.



Puerto / componente	Responsabilidad	Adaptador local
DatasetSource	Adquirir datos desde una fuente y devolver el modelo común.	Archivos locales, PostgreSQL y SQL Server
SecretStore	Gestionar credenciales por organización, fuera de artifacts.	Cifrado Fernet local, clave separada
DatasetReader	Interpretar un formato de archivo y sus opciones de lectura.	CSV, XLSX, JSON, Parquet, TXT
StorageProvider	Publicar bytes inmutables, materializar un artefacto y asignar staging temporal.	FileArtifactStore
ExecutionEngine	Ejecutar un Run completo con evidencia y linaje.	LocalExecutionEngine
ProcessingEngine	Compilar/evaluar expresiones portables de reglas y comparaciones.	Polars; DuckDB para paridad cubierta
ExecutionPlanner	Estimar recursos y fijar un plan antes de ejecutar.	Presupuesto de memoria/disco local
JobQueue	Publicar el trabajo asociado a un Run.	DatabaseJobQueue

Las reglas no reciben cadenas de conexión ni conocen CSV, Excel o buckets. Trabajan sobre el modelo común producido por los lectores y el Parquet canónico. La orquestación usa identidades de versiones y configuraciones, genera hallazgos y publica evidencia mediante los puertos.

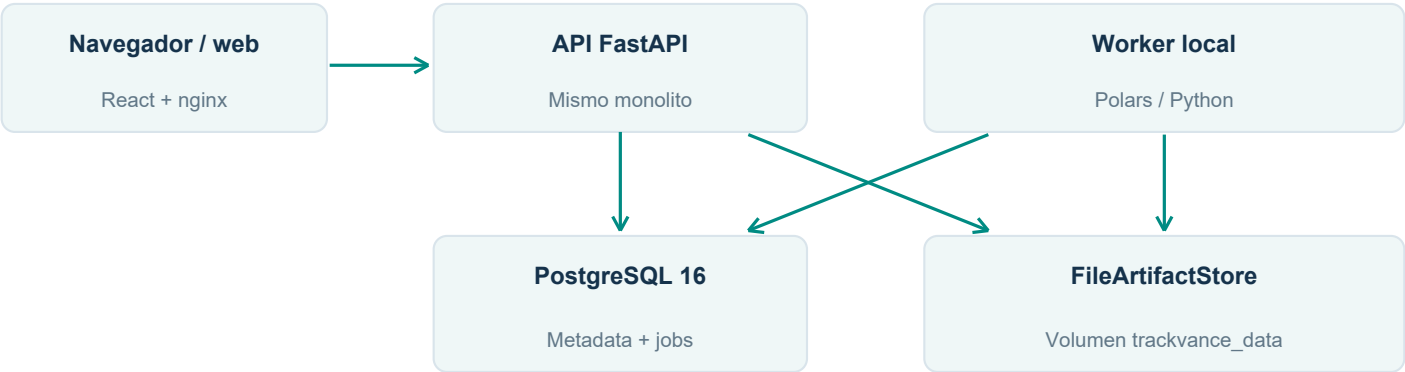
Separación incremental

La implementación mantiene un paquete Python compacto. API, servicios, modelos ORM, reglas, adapters y worker están en módulos distintos, pero aún no existe una capa de dominio totalmente libre de ORM. ExecutionEngine y JobQueue reciben Session/Run de SQLAlchemy. Esta frontera es suficiente para sustituir infraestructura de forma progresiva; no se declara una arquitectura hexagonal pura ni una sustitución cloud inmediata sin trabajo adicional.

StorageProvider devuelve Path únicamente para staging/materialización necesarios por los motores locales. Los servicios no deciden el directorio del volumen. Los campos históricos *_path permanecen compatibles como referencias internas; una migración futura de locators debe preservar IDs, hashes y linaje.

3. Despliegue local de referencia

IMPLEMENTADO. Docker Compose es el entorno de referencia para operación y certificación con PostgreSQL. El arranque directo Windows/SQLite se conserva como comodidad de desarrollo y usa una instalación de datos independiente.



Servicio	Función	Persistencia / red
web	React compilado servido por nginx y proxy a /api.	Publicado solo en 127.0.0.1; puerto configurable.
api	FastAPI, sesión/RBAC, carga, configuración, consulta y downloads.	Red interna Compose; comparte trackvance_data.
worker	Procesa trabajos del mismo monolito, leases y heartbeat.	Red interna; comparte metadata y trackvance_data.
postgres	PostgreSQL 16 para metadata.	Volumen postgres_data; sin puerto publicado al host.

El volumen trackvance_data se monta en /var/lib/trackvance para API y worker. FileArtifactStore administra artifacts/<id>/data.<ext> y staging tmp; las rutas históricas permanecen legibles. No se guardan datasets como blobs SQL. Los volúmenes sobreviven a stop, restart, recreación de contenedores y docker compose down. down -v elimina volúmenes y no es un comando de actualización.

Solo API monta connection_credentials y connection_keys para adquirir fuentes externas. El worker monta únicamente trackvance_data: consume snapshots canónicos sin recuperar credenciales de conexión. La copia de recuperación debe conservar metadata, artifacts, credenciales cifradas y clave de forma coordinada.

Todos los servicios declaran restart: "no". Trackvance permanece detenido al iniciar Docker Desktop hasta que el usuario ejecute el arranque. La instancia del equipo usa trackvance-certification y http://localhost:3100; el valor por defecto del repositorio es 3000.

```
docker compose up --build -d --wait
docker compose stop
docker compose up -d --wait
docker compose ps
```

La preparación descarga imágenes y dependencias. Una vez construidas, la operación no requiere fuentes, storage, identidad ni procesamiento externos. El override de red interna aísla API/worker/PostgreSQL; la salida de nginx depende de Docker Desktop/firewall y no se presenta como aislamiento total del host.

4. Código, stack y dependencias

Capa	Implementación actual
Frontend	React 19, TypeScript, Vite, React Router, TanStack Query, Radix Dialog, Lucide y CSS propio.
HTTP / persistencia	FastAPI, Pydantic, SQLAlchemy 2, Alembic, psycpg y PostgreSQL 16.
Procesamiento	Polars/Python, Decimal, expresiones portables y compilador DuckDB.
Formatos / evidencia	openpyxl para XLSX, lectores locales, Parquet, JSON y SHA-256.
Pruebas	pytest, Ruff, Mypy, ESLint, TypeScript, Vitest, Testing Library y Playwright.
Reproducibilidad	backend/uv.lock, frontend/pnpm-lock.yaml, Dockerfiles y compose.yml.

Responsabilidades del backend

Archivo	Responsabilidad principal
api.py	DTOs HTTP, sesiones, autorización, errores y endpoints.
services.py	Casos de uso: versiones, runs, evidencia, hallazgos y validación de excepciones.
models.py / db.py	Modelo persistente y unidad de sesión SQLAlchemy.
processing.py / config_semantics.py	Semántica de perfil, reglas, normalización y resultados.
portable_engine.py / planner.py	Compilación portable, evaluación y preflight.
dataset_readers.py	DatasetSource, registry y lectores por formato.
connections_api.py / connections_service.py	Contratos HTTP, configuración versionada, bindings y adquisición de fuentes.
dataset_sources.py / credential_store.py	Adaptadores SQL de solo lectura y contrato/proveedor de secretos.
artifactstore.py	StorageProvider y adaptador local inmutable.
execution.py / jobqueue.py / worker.py	Puerto de ejecución, publicación, consumo y recuperación local.
manifests.py / exports.py	Compatibilidad de evidencia y reportes Excel.
permissions.py / audit_context.py	Política RBAC, actor y sanitización de metadata.
dashboard.py	Agregación operativa del Centro de Control.

El frontend se organiza en app, components, features/datasets, features/connections, features/runs y routes; comparte cliente HTTP, componentes visuales y queries. La organización por bounded contexts/domain/application/infrastructure podrá profundizarse cuando el tamaño lo justifique. No se mueve código sin un beneficio funcional o de frontera concreto.

No forman parte del stack instalado React Hook Form, Zod, Tailwind, TanStack Table, Recharts, PyArrow ni PySpark. La documentación inicial que los proponía se interpreta como antecedente de diseño, no como inventario actual.

5. Fuentes y lectores de datasets

IMPLEMENTADO. `DatasetSource.read(options, inspect)` devuelve `DatasetReadResult`. `LocalFileDatasetSource` resuelve el lector en `DatasetReaderRegistry`. Cada lector interpreta el formato sin aplicar transformaciones de negocio.

`DatasetReadResult` contiene `frame`, `source_format`, `format_label`, `media_type`, `row_numbering`, `native_schema`, hojas/opción seleccionada, delimitador detectado, `row_count` y metadata acotada. El frame común representa escalares observados como `String/Null`. La metadata nativa informa la inferencia sin alterar la representación recibida.

Fuente local	Detección y comportamiento
CSV	UTF-8 con BOM opcional; coma/punto y coma conservan compatibilidad. Conserva líneas físicas, incluidos registros multilinea.
XLSX	Verifica paquete Excel, limita expansión y rechaza macros. Enumera hojas, permite seleccionar y usa lectura acotada. Las fórmulas se tratan como texto observado.
JSON	Arreglo de objetos, objeto individual, objeto con un único arreglo tabular y JSONL/NDJSON. Objetos anidados simples se aplanan con rutas de punto; listas internas permanecen JSON compacto.
Parquet	Usa esquema embebido y footer. Revisa filas, columnas y expansión antes de leer; la inspección no requiere decodificar todo el archivo.
TXT / TSV	Detección de coma, punto y coma, tabulación o barra vertical; permite fijar un carácter ASCII seguro.

La resolución usa firmas fuertes, contenido, extensión y detección delimitada cuando procede. MIME y extensión de descarga se derivan del formato verificado; un nombre engañoso no determina por sí solo el parser ni el tipo de descarga.

Límites locales

La carga admite por defecto 10 MiB, 100.000 filas y 100 columnas. La inspección previa usa hasta 100 registros y metadata embebida donde existe. Los lectores limitan expansiones XLSX/Parquet y tamaño de celdas. La inspección controla la muestra; la carga/perfil inicial completos siguen siendo síncronos y acotados.

Conectores externos

`PostgreSQLDatasetSource` y `SQLServerDatasetSource` están implementados. Resuelven credenciales mediante `SecretStore`, limitan consultas en origen, conservan metadata reproducible sin secretos y producen el modelo común. S3, Azure Blob, REST, Oracle, MySQL, Snowflake y Databricks son futuros adaptadores. La adquisición podrá ejecutarse en un job sin cambiar las reglas.

5A. Módulo Conexiones

IMPLEMENTADO. Permite crear, probar, editar, deshabilitar y eliminar lógicamente conexiones PostgreSQL y SQL Server. La conexión registra nombre, motor, host, puerto, base, usuario y opciones controladas. La contraseña se envía únicamente al guardar/probar y nunca vuelve en una respuesta. El formulario exige una prueba vigente y backend vuelve a verificar antes de guardar. La deshabilitación puede hacerse aunque la fuente esté caída.

Después de conectar, el usuario explora schemas, tablas y vistas accesibles. La vista previa muestra nombres, tipos nativos/lógicos, nulabilidad y hasta 100 registros. La detección del esquema usa catálogos; no escanea la tabla completa. Registrar la selección crea un Dataset, su binding de origen y una DatasetVersion canónica Parquet mediante StorageProvider. Los motores de calidad siguen leyendo versiones locales inmutables.

Límites de responsabilidad

El PostgreSQL interno guarda solo metadata del sistema. PostgreSQL/SQL Server externos son fuentes del usuario y tienen credenciales independientes. Ningún adaptador escribe filas en la fuente. No se recibe SQL arbitrario; se seleccionan identificadores previamente descubiertos y citados según el motor. PostgreSQL usa transacciones read-only. En SQL Server la intención read-only es informativa: es obligatorio usar una cuenta con permisos SELECT mínimos, sin permisos de escritura.

Las fuentes externas requieren acceso de red desde API. El overlay offline restringe esa salida; el Compose base permite conexión a fuentes accesibles, sin añadir dependencias cloud. Para fuentes en el PC bajo Docker Desktop puede usarse host.docker.internal. La configuración local conserva restart=no.

Representación y límites

El preview aplica LIMIT/TOP en origen; el snapshot lee por lotes de 100 hasta 100.000 filas, 100 columnas y 64 MiB de valores normalizados. Cada celda tiene límite de 64 KiB. Una fila adicional detecta el exceso y se rechaza la importación completa: nunca se guarda una muestra parcial como si fuera un dataset completo. Timeout configurable de conexión 1..15 segundos; consulta 1..60 segundos. La adquisición sigue siendo síncrona y acotada.

Los valores se conservan como texto/null: Decimal sin redondeo float, fechas ISO, Unicode y espacios originales. Los identificadores siguen la política id/*_id y permiten override. Los binarios son Base64. SQL Server timestamp se interpreta como rowversion; datetimeoffset se conserva como ISO con offset. El orden DATABASE_UNSPECIFIED depende del motor; SNAPSHOT_ROW es posición en el snapshot, no un número físico en la tabla externa.

Versiones y evidencia

La conexión tiene ID estable y revisión para concurrencia optimista. Cada edición crea ExternalConnectionVersion inmutable con hash de configuración y referencia opaca de credencial. Un binding conserva dataset, conexión, schema, tabla/vista y overrides. Actualizar desde fuente usa la configuración vigente y crea una versión nueva. Fallos de acceso no modifican versiones anteriores.

ingestion_metadata.source conserva connection_id, connection_version_id, connection_version, source_type, schema_name, object_name, object_kind, config_hash y captured_at. El linaje SOURCE_SNAPSHOT apunta a la revisión de conexión; REFRESH_OF apunta al snapshot anterior. Los manifests de runs incluyen la identidad de fuente de cada DatasetVersion. El run original y su hash permanecen inmutables aunque cambie la conexión.

SecretStore, permisos y auditoría

El contrato put/get/delete permite sustituir el proveedor local por Azure Key Vault, AWS Secrets Manager o HashiCorp Vault. La implementación local cifra con Fernet autenticado, vincula credencial a organización/referencia y persiste en connection_credentials; la clave maestra reside en connection_keys. En Linux los directorios son 0700 y archivos 0600; Windows directo hereda ACL del usuario. El backup debe proteger ambos volúmenes por separado del almacenamiento de artifacts. Una clave perdida no se reemplaza silenciosamente si ya existen secretos. Rotación automática de clave maestra permanece pendiente.

La contraseña guardada solo se reutiliza en pruebas o ediciones cuando host, puerto, base, usuario y modo TLS coinciden con la versión vigente. Cambiar cualquiera exige una contraseña explícita; su ausencia devuelve 422

PASSWORD_REQUIRED_FOR_ENDPOINT_CHANGE antes de contactar el destino. Nombre y timeouts pueden cambiar conservando el secreto. Esto impide redirigir una credencial guardada a otro servidor desde un formulario de edición.

Transporte cifrado por defecto: PostgreSQL sslmode=require y SQL Server encryption=require. PostgreSQL permite verify-ca/verify-full con CA configurada. Requerir cifrado no equivale a verificar identidad del servidor; la confianza de certificados

debe administrarse en el driver. Los modos disable/off se reservan para pruebas locales. FreeTDS comparte timeouts a nivel proceso; el adaptador SQL Server serializa conexiones para evitar carreras.

RBAC separa connections:read, connections:use y connections:manage. Administradores y responsables de datos administran; Analista puede usar. Los roles de consulta solo ven metadata. Todas las rutas verifican organización. La auditoría registra creación, edición, prueba, preview, baja y adquisición con actor estable; nunca password, DSN ni filas de preview. Los errores de drivers se convierten a códigos controlados y mensajes constantes.

Fuentes futuras y Data Delivery

Un nuevo adaptador implementa DatasetSource y su normalización, registra opciones permitidas y aporta pruebas de permisos, límites, tipos y trazabilidad. Fuentes no SQL tendrán DTOs específicos, sin forzar el concepto de schema/tabla sobre buckets o APIs. El motor de calidad no cambia. Un futuro módulo Destinos / Data Delivery tendrá un puerto independiente de escritura, RBAC e idempotencia propios. DatasetSource no incorpora write ni entrega hacia sistemas externos.

6. Carga, esquema y versionado

El flujo previo a guardar una versión detecta formato, hoja/delimitador, columnas y tipos. El usuario puede corregir el tipo lógico o marcar identificadores. Los mismos controles se ofrecen al crear una versión de un dataset existente.

- Columnas identificadoras permite escribir nombres o seleccionarlos del esquema, con opción Todos.
- Los tipos disponibles son STRING, INT64, DECIMAL, DATE, TIMESTAMP y BOOLEAN; un override explícito se valida contra los valores del archivo.
- El campo Área de negocio permite agregar un área nueva. El listado muestra origen entre Dataset y Área: Manual para carga, Data Intake para salida derivada.
- El listado combina búsqueda/filtro y orden ascendente/descendente por Área, Registros, Versiones, Estado y Última actualización.
- Un nombre ya usado en la organización devuelve DATASET_NAME_EXISTS y permite orientar al usuario hacia una versión nueva.

Secuencia persistente

1. La API escribe el upload en staging limitado del provider y detecta el lector.
2. El lector produce el modelo común; profiling valida inferencia/overrides.
3. StorageProvider conserva ORIGINAL_UPLOAD con hash, tamaño y MIME.
4. Se publica CANONICAL_PARQUET y se crea DatasetVersion inmutable con esquema, perfil e ingestion_metadata.
5. ArtifactLink registra relaciones y AuditEvent registra DATASET_UPLOADED. Los temporales del flujo normal se eliminan.

DatasetVersion conserva dataset_id, version, filename, source_type, SHA-256, schema_hash, filas/columnas, profile_status, schema_json, profile, ingestion_metadata, parent_version_id, original_artifact_id, canonical_artifact_id y source_run_id. original_path/canonical_path se conservan como metadata interna compatible, no se exponen como rutas del cliente.

ingestion_metadata incluye reader.key/version, source_format, format_label, reader_options efectivas, esquema nativo y política de numeración. Los históricos reciben un objeto vacío y siguen el adaptador compatible sin recalculer sus perfiles.

GET /datasets/{id}/schema usa por defecto el perfil persistido. refresh=true vuelve a leer el footer canónico y conserva tipos lógicos declarados; devuelve scan_mode/scanned_rows y no aplica transforms ni reescribe historia. El control SHA-256 puede leer bytes para integridad aunque no procese filas de negocio.

7. Profiling, identificadores y transforms

La política actual es EXACT_OBSERVED, metric_definition_version=2. No se recortan espacios, cambia case ni normaliza Unicode para contar valores distintos. "Cliente 12" y " Cliente 12 " son valores distintos; las diferencias de mayúsculas y de representación Unicode se conservan. Null y texto vacío tienen semánticas propias del lector y del perfil; no se equiparan mediante un trim implícito.

Inferencia inequívoca

DATE se infiere cuando los valores no nulos válidos cumplen YYYY-MM-DD y representan fechas reales. Una fecha inválida o una mezcla incompatible impide inferir DATE. TIMESTAMP exige su semántica ISO diferenciada, con zona explícita según las reglas portables; no se adivinan fechas ambiguas como 01/02/2026.

IDENTIFIER tiene prioridad sobre la interpretación como magnitud. Los nombres id y *_id activan una heurística documentada; el usuario puede confirmar/override el esquema. "001234567" conserva su cero inicial frente a "1234567". No se convierten universalmente todas las columnas numéricas a identificadores. El esquema nativo Parquet ayuda a inferir tipos, incluso con columnas nulas, pero no invalida el override explícito.

Los perfiles incluyen conteos de nulos y distintos, tasas y razón de unicidad, además de método/versión. Las series históricas LEGACY_NORMALIZED/1 se mantienen separadas de EXACT_OBSERVED/2 para no comparar métricas semánticamente incompatibles.

Transformaciones funcionales

Intake admite una lista ordenada de transforms {column,type,parameters}: trim, case, unicode_normalization, empty_to_null, id_padding, remove_characters, decimal_parse y date_parse. Se ejecutan únicamente cuando el contrato las declara y afectan al output de esa ejecución. El upload original y su perfil histórico no se reescriben.

Las configuraciones nuevas se validan y normalizan a schema_version=2. configuration_hash usa JSON determinista con claves ordenadas y SHA-256; stored_config_hash conserva además la identidad de la configuración almacenada. La adaptación legacy se centraliza en config_semantics y manifests, evitando condiciones de versión dispersas en módulos.

8. Reglas portables y Data Intake

RuleDefinition declara type, column, severity ERROR/WARNING, enabled, parameters, code opcional y message opcional. El código ausente se deriva de type; un código personalizado debe ser estable. La declaración se compila a una representación portable antes de evaluarse con el engine. No se admite scripting arbitrario ni Python eval/exec.

Tipo / código	Parámetros y semántica
required / REQUIRED	Campo presente, no null y no vacío. Whitespace no es vacío sin trim declarado.
not_null / NOT_NULL	Rechaza null.
unique / UNIQUE	Incumplen todos los registros duplicados; null_policy explícita.
numeric / NUMERIC	Decimal finito con punto; conserva precisión.
positive / POSITIVE	Decimal estrictamente mayor que cero.
type / TYPE	logical_type STRING, INT64, DECIMAL, DATE, TIMESTAMP o BOOLEAN.
range / RANGE	gt/gte y lt/lte, límites Decimal inequívocos.
allowed_values / ALLOWED_VALUES	values con hasta 1000 escalares; pertenencia exacta.
regex / REGEX	Subconjunto portable Rust/RE2, flags i/m/s, máximo 500 caracteres.
date_rule / DATE_RULE	not_future, min/max ISO, timezone UTC y null_policy.

null_policy admite ALLOW, FAIL o IGNORE según la regla. Los shorthand históricos numeric/positive conservan FAIL. El instante de referencia de fecha se fija al inicio del Run para que una misma ejecución no cambie de criterio entre registros.

Editor y resultados

Al elegir dataset, el editor carga su esquema. Columnas obligatorias, sin duplicados, numéricas y positivas usan multiselect con Todos; las positivas sólo permiten columnas numéricas. El sistema detecta candidatos, pero el usuario decide las reglas. El esquema puede actualizarse desde el formulario. Las reglas avanzadas se construyen mediante los controles disponibles o mediante la API documentada; no todas las opciones de la DSL tienen editor visual completo.

Intake devuelve total_rows, valid_rows, error_rows, warning_rows, acceptance_rate, resultados por regla y decisión APPROVED/APPROVED_WITH_WARNINGS/REJECTED según severidad y max_error_rate. Una fila puede producir varios errores; error_rows cuenta registros afectados, no la suma de mensajes.

El output reutilizable se guarda como Parquet INTAKE_ACCEPTED, con source_type=INTAKE_OUTPUT, canonical_artifact_id, parent_version_id y source_run_id. No tiene upload original ficticio. La UI muestra Artefacto derivado/Fuente de la versión y el linaje enlaza el input y el IntakeRun.

9. ReconOps y normalización declarada

ReconOps compara versiones inmutables de origen y destino. `key_columns` admite una o varias columnas. El control conserva `key_normalization` en su versión, `config hash`, `manifest`, `plan` y `diagnostics`.

```
{"trim": false, "case": "NONE", "unicode_normalization": "NONE"}
```

`case` admite NONE/UPPER/LOWER y `unicode_normalization` NONE/NFC/NFKC. Los controles nuevos usan NONE por defecto. Los históricos sin el campo conservan el comportamiento legacy de trim mediante adaptación versionada; no se cambia silenciosamente el resultado de runs anteriores.

Regla	Semántica
EXACT_COMPARE	Igualdad textual exacta tras la normalización declarada; admite múltiples pares de columnas y <code>equal_nulls</code> .
NUMERIC_TOLERANCE	Comparación Decimal con <code>abs</code> y <code>percent</code> opcional; denominador SOURCE/TARGET/MAX_ABS y política de denominador cero EXACT_ONLY. Umbral inclusivo.
DATE_TOLERANCE	Diferencia temporal contra hours o days, con zona UTC y <code>equal_nulls</code> .
AGGREGATE_COMPARE	Capacidad 1:N simple por claves con <code>sum</code> o <code>count</code> sobre un lado; después se aplican <code>comparison_rules</code> sobre <code>output_column</code> .

EXACT_MATCH se conserva como alias explícito legacy de NUMERIC_TOLERANCE. Una tolerancia numérica cero sigue comparando magnitudes: "01" y "1" pueden coincidir. Esto es distinto de EXACT_COMPARE. `amount_column/tolerance` históricos se adaptan sin modificar el snapshot.

Clasificaciones estables

MATCH, VALUE_MISMATCH, SOURCE_ONLY, TARGET_ONLY, DUPLICATE_SOURCE, DUPLICATE_TARGET e INVALID. Las filas duplicadas no se convierten en coincidencias arbitrarias. Las comparaciones conservan valores, regla, diferencia, tolerancia y líneas origen/destino; agregaciones conservan las líneas de sus integrantes.

`metrics` incluye `source_rows/target_rows`, `matched`, `mismatched`, `faltantes`, `duplicados`, `invalid`, `total_rows`, `match_rate` y `counts` por clasificación. La decisión es CONFORME o WITH_FINDINGS. Los hallazgos agrupan categorías distintas de MATCH; la evidencia conserva el detalle verificable de cada resultado.

La precisión monetaria se implementa con Decimal y contexto suficiente para sumas, diferencias y porcentajes. Las expresiones tienen pruebas de paridad Polars/DuckDB donde se declara soporte. No se infiere equivalencia de todo el pipeline entre motores a partir de estas pruebas.

10. Sentinel y métricas históricas

Sentinel ejecuta monitores sobre una DatasetVersion y produce checks explicables. Los controles legacy SCHEMA_REQUIRED, NULL_RATE, FRESHNESS y VOLUME_CHANGE permanecen compatibles. Una columna ausente se distingue de una columna presente con nulls.

Control	Parámetros / cálculo
SCHEMA_REQUIRED	Lista de columnas obligatorias del monitor.
NULL_RATE	Columnas observadas y máximo permitido.
FRESHNESS	Edad respecto a fecha de versión y max_age_hours.
VOLUME_CHANGE	Cambio porcentual respecto a versión anterior.
DISTINCT_COUNT	Columna y min/max de valores distintos.
DISTINCT_RATE	Columna y límites en fracción 0..1.
UNIQUENESS_RATIO	Proporción de registros únicos, con límites 0..1.
SCHEMA_TYPE	Tipo esperado o comparación con el esquema de la versión anterior.
METRIC_THRESHOLD	Métrica/columna, operador y threshold o min/max.
HISTORICAL_BAND	window, min_history, iqr_multiplier y fallback explícito.

Las reglas por registro de Intake también pueden evaluarse como checks de Sentinel con conteos evaluados/fallidos. El resultado indica code, name, status, actual, expected y message. La salud es la fracción de checks aprobados; cero fallos da HEALTHY y cualquier fallo da ALERT.

Compatibilidad de series

SentinelMetricHistory conserva run_id, monitor_id, metric_key, dimensiones y hash, numeric_value, method, metric_definition_version y observed_at. Sólo se comparan ejecuciones SUCCESS con método/versión compatibles. El historial no mezcla perfiles legacy normalizados con observaciones nuevas exactas.

La banda histórica utiliza mediana e IQR con el método MEDIAN_IQR_LINEAR_V1. Cuando no hay historia suficiente, el resultado identifica FIXED_THRESHOLD_FALLBACK o PREVIOUS_COMPATIBLE_METRIC según la política. El expected muestra qué método se aplicó; no inventa una referencia estadística.

El monitor puede detectar schema drift sin que el worker falle técnicamente. El Run original conserva ALERT aunque una versión posterior vuelva a estar saludable. Las excepciones asociadas sólo se validan cuando una ejecución posterior del mismo snapshot de monitor termina HEALTHY.

11. Excepciones con validación técnica

Una excepción nace de un Finding y conserva finding_id, run_id de origen y configuration_id del snapshot que lo produjo. El Run de origen nunca cambia su decisión porque un caso se cierre o los datos se corrijan.



Run posterior SUCCESS de la misma configuración

La evidencia técnica habilita Resolver; cierre administrativo exige motivo

Estado	Función
OPEN / Abierta	Hallazgo convertido en caso, pendiente de gestión.
INVESTIGATING / En gestión	Responsable investiga causa y prepara corrección.
PENDING_VALIDATION	Espera evidencia técnica posterior.
RESOLVED / Resuelta	Evidencia válida, causa y resolución documentadas.
DISCARDED / Descartada	Cierre administrativo con motivo obligatorio.
ACCEPTED / Aceptada	Riesgo o situación aceptada con motivo obligatorio.
NOT_APPLICABLE / No aplica	Cierre administrativo justificado.

Criterios de elegibilidad

Se requiere un Run posterior SUCCESS, de la misma organización y el mismo configuration_id. Publicar otra versión de configuración crea otro snapshot y no valida automáticamente la excepción anterior. Se selecciona la candidata elegible más reciente; no se acepta una validación positiva obsoleta cuando existe una ejecución posterior que vuelve a fallar.

- Intake: debe pasar la regla/columna que originó el hallazgo.
- ReconOps: el control debe quedar conforme o desaparecer la clasificación asociada al hallazgo.
- Sentinel: el monitor debe volver a HEALTHY.

La UI deshabilita Resolver mientras no se cumplen los criterios y muestra el motivo. Una validación correcta se presenta como Validada técnicamente y enlaza el Run confirmatorio. Resolver sólo se acepta desde PENDING_VALIDATION y exige causa raíz y resolución. La API reevalúa antes de guardar.

El worker actualiza evidencia de casos PENDING_VALIDATION al terminar un Run. Conserva validation_run_id, validated_at, validation_evidence y evento TECHNICAL_VALIDATION. Esto prepara una política futura de resolución automática; hoy la resolución final sigue siendo una acción humana.

Las mutaciones usan version para concurrencia optimista. Timeline conserva actor estable, fecha, estado anterior/nuevo, comentario y referencia de validación. Los cierres administrativos no equivalen a RESOLVED ni fabrican evidencia. WAITING_EXTERNAL/FALSE_POSITIVE y resoluciones históricas permanecen legibles; los registros anteriores sin prueba se identifican como históricos, sin inventar un Run confirmatorio.

12. Centro de Control y navegación

El Centro de Control es un cockpit operativo calculado por backend. El usuario puede identificar qué falla, qué requiere atención y abrir la acción correspondiente. Las cifras no son constantes de presentación.

Elemento	Comportamiento actual
Filtros globales	Período 7d/30d/90d/all, dataset, módulo, estado operativo y criticidad.
Indicadores superiores	Salud general, controles fallidos, excepciones abiertas y datasets afectados. Variación contra período anterior cuando existe una comparación válida.
Requiere tu atención	Excepciones, hallazgos y fallos priorizados; muestra dataset, módulo, problema, fecha y acción directa.
Salud de los datos	Evolución temporal y desglose Intake/Recon/Sentinel.
Datasets afectados	Salud, hallazgos, última ejecución y tendencia.
Resumen por módulo	Ejecuciones/controles/monitores con fallos y alertas.
Ejecuciones recientes	Dataset, registros, hallazgos y duración cuando están disponibles; estado técnico separado de decisión.
Accesos rápidos	Carga de dataset y ejecución de operaciones.

health_score se pondera por unidades evaluadas según la agregación del backend. Las variaciones devuelven previous/delta o null cuando no hay base válida. Los filtros respetan organización y los enlaces navegan al recurso correspondiente. No se presenta ausencia de historia como mejora o deterioro.

Rutas SPA

/; /datasets; /datasets/:id; /intake/*; /recon/*; /sentinel/*; /runs; /runs/:id; /exceptions; /rules; /audit; /settings/*. El menú conserva los módulos y el diseño navy/teal. Estados de carga, vacío y error tienen tratamiento específico; se conserva request_id en errores para diagnóstico.

Los detalles de dataset separan archivo original, fuente derivada, esquema/perfil y linaje. Los detalles de run separan Completada de Rechazado/Con hallazgos/Alerta. Los labels de negocio acompañan reason codes técnicos en resultados y evidencia. La numeración se muestra como Línea del archivo cuando representa línea física o Registro de la versión cuando corresponde.

13. StorageProvider, artefactos y linaje

StorageProvider es el puerto de almacenamiento interno. `temporary_path` asigna staging local temporal; `put_file` publica bytes y metadata inmutables; `materialize/materialize_reference` obtiene un archivo verificable para el motor local; `open_read/exists` completan el acceso. Un adaptador remoto deberá administrar su cache materializado y su ciclo de limpieza.

FileArtifactStore confina las rutas al root configurado, verifica tamaño/SHA-256 y publica sin reemplazar bytes históricos. El nombre recibido nunca se usa como path arbitrario. Los servicios nuevos no construyen el layout del volumen; el adaptador conserva registro directo/promote para compatibilidad y backfill.

Kind	Uso
ORIGINAL_UPLOAD	Archivo realmente recibido del usuario.
CANONICAL_PARQUET	Representación canónica de una versión cargada.
INTAKE_ACCEPTED	Parquet de filas aceptadas; base reutilizable de versión derivada.
INTAKE_ERRORS	Detalle de incumplimientos Intake.
RECON_RESULTS	Resultados detallados de conciliación.
SENTINEL_PROFILE	Checks/resultados del monitor.
RUN_MANIFEST	Evidencia JSON de ejecución.
EXPORT_XLSX	Reporte estructurado generado bajo demanda.
GENERATED_DEMO / legacy	Datos ficticios o artefactos históricos identificados explícitamente.

Artifact guarda `id`, `organization_id`, `kind`, `name`, `path` interno, `sha256`, `size_bytes`, `media_type` y fecha. ArtifactLink registra relaciones dirigidas `DERIVED_FROM`, `RUN_INPUT`, `RUN_OUTPUT`, `INTAKE_ACCEPTED_FROM` y `EXPORT_OF` entre versiones, runs y artefactos.

Antes de procesar o descargar, se comprueba la identidad registrada y su integridad.

`ARTIFACT_HASH_MISMATCH/ARTIFACT_INTEGRITY_ERROR` impide usar bytes corruptos; no recalcula silenciosamente el hash histórico. El backfill aditivo registra archivos legacy verificables, cuenta ausentes/corruptos y conserva historia.

Una publicación interrumpida puede dejar bytes huérfanos no referenciados después de un rollback. Esos bytes no reemplazan artefactos válidos. Retención, recolección de huérfanos y políticas de object storage son evolución pendiente y deben implementarse con referencias transaccionales y auditoría.

14. Manifest schema 2 y auditoría

Cada Run completado publica un manifest inmutable con `schema_version=2`. `read_manifest/adapt_manifest` centraliza la lectura de v1 histórico; la adaptación se realiza en memoria, sin sobrescribir archivos anteriores.

```
{
  "schema_version": 2,
  "run_id": "<id>",
  "module": "recon",
  "initiated_by": {
    "type": "USER", "id": "<id estable>",
    "display_name": "Equipo Trackvance"
  },
  "result_artifacts": [{
    "artifact_id": "<id>", "kind": "RECON_RESULTS",
    "name": "results.parquet", "sha256": "<hash>",
    "size_bytes": 1234
  }]
}
```

El contrato completo incluye `started_at/finished_at`, identidad/legacy, versión de Trackvance, plan processing con `engine/version`, inputs con `DatasetVersion/SHA/schema hash/origen/ingestion_metadata`, configuración efectiva/`id/version/config_hash/stored_config_hash`, métricas, `output_version_id` y `result_artifacts`.

La versión del producto y la del motor se distinguen: `engine_version` de nivel superior identifica Trackvance en el manifest actual; `processing.engine_version` identifica la versión real de Polars. La UI y exports muestran ambas con su contexto.

Identidad y eventos

`AuditEvent` conserva `actor_type`, `actor_id` estable, `actor_visible`, `actor_legacy`, `event_type`, `subject_type/id`, `request_id`, `run_id`, metadata sanitizada y timestamp. Actor puede representar `USER`, `WORKER` o `SYSTEM`. Los nombres visibles no sustituyen la identidad persistente.

`DATASET_UPLOADED`, `DATASET_DERIVED`, `CONFIGURATION_PUBLISHED`, `RUN_QUEUED`, `RUN_COMPLETED`, `EXCEPTION_CREATED`, `EXCEPTION_UPDATED` y `EXCEPTION_VALIDATION_CHECKED` documentan el ciclo. `EXPORT_DOWNLOADED`, `EVIDENCE_DOWNLOADED` y `ARTIFACT_DOWNLOADED` registran el acceso relevante a evidencia. La UI enlaza recursos desde los eventos.

Se excluyen claves de passwords, tokens y secretos de metadata/evidencia. Los casos legacy sin identidad demostrable se marcan como legacy; no se atribuyen a un UUID de usuario inventado. Las auditorías de exportación agregan evidencia sin alterar el manifest original del Run.

15. Reportes Excel de negocio

IMPLEMENTADO. La UI ofrece Exportar Excel y genera .xlsx mediante openpyxl. CSV permanece únicamente como endpoint deprecated para consumidores históricos. El Excel es un artefacto derivado; Parquet sigue siendo canónico.

Módulo	Hojas y contenido
Intake	Resumen: contrato/version, dataset/version, run, fechas, estado/decisión, filas/tasas/umbral/motor y reglas. Errores: línea, regla, columna, valor, severidad, mensaje y clasificación. Reglas: configuración efectiva. Trazabilidad: IDs/hashes/actor/plan/artefactos.
ReconOps	Resumen: control, ambos inputs, claves/comparaciones/tolerancias, siete conteos y match rate. Resultados: clave, clasificación, valores, diferencia, tolerancia, líneas, mensaje y comparaciones. Hallazgos: no MATCH. Trazabilidad: ambos inputs y evidencia.
Sentinel	Resumen: monitor, versión, filas, checks/fallos, salud y decisión. Controles: código/estado/observado/esperado/detalle. Hallazgos: sólo fallidos. Trazabilidad: input, configuración, motor y artefactos.

El diseño usa encabezados navy, teal para acentos, rojo para incumplimientos y amber para warning. Tablas con autofiltro, paneles inmovilizados, anchos legibles, ajuste de texto y formatos de fecha, porcentaje y decimal. Las columnas técnicas se concentran en Trazabilidad y los reason codes siguen disponibles.

Tipos y seguridad

Identificadores se escriben como texto. Decimales de más de 15 cifras significativas también se conservan como texto para evitar pérdida de precisión de Excel. Los valores de negocio que comienzan con =, +, -, @, tabulación o espacios nunca se convierten en fórmula ejecutable ni link externo. Las reglas de formato condicional son constantes internas.

No se trunca evidencia silenciosamente: celdas mayores que 32767 caracteres o tablas que exceden límites de Excel producen EXPORT_LIMIT_EXCEEDED. Controles XML inválidos se escapan de forma visible. Passwords/tokens/secrets no se incluyen.

GET /runs/{id}/export.xlsx exige sesión, exports:download, organización, SUCCESS e integridad de manifest/resultados. El MIME es application/vnd.openxmlformats-officedocument.spreadsheetml.sheet; Content-Disposition usa trackvance_<module>_<run>.xlsx sanitizado y X-Artifact-ID identifica el export. Se registra EXPORT_XLSX/EXPORT_OF y auditoría.

16. Ejecución, planificación y recuperación

POST de ejecución valida configuración/versions, fija actor e inputs inmutables, calcula preflight y crea Run más Job en la misma sesión. Idempotency-Key evita crear runs duplicados para el mismo endpoint/organización/key/payload; cambiar el payload con la misma key produce IDEMPOTENCY_CONFLICT.

Estado técnico	Significado
QUEUED	Trabajo persistido esperando worker.
RUNNING	Worker posee lease y procesa la ejecución.
SUCCESS	Procesamiento terminado y evidencia publicada; decisión de negocio separada.
FAILED	Error técnico durante ejecución.
FAILED_PRECONDITION	Preflight no permite ejecutar; no se simula resultado.
CANCELLED	Cancelación atendida por el worker.

DatabaseJobQueue publica Jobs con run_id único. El worker reclama trabajos elegibles con actualización condicional, identifica propietario y lease_until, registra intentos y recupera leases vencidos. Heartbeat informa actividad local y salud. Antes de publicar resultados se revalida lease; los intentos no sobrescriben evidencia histórica.

ExecutionPlanner

Estima memoria de trabajo desde bytes/filas/columnas y un factor mayor para Recon. Comprueba disco temporal libre. Los defaults son 512 MiB de presupuesto blando, 16 MiB de reserva mínima de disco y 300 segundos en resource_budget. Estos valores son configurables por entorno.

El pipeline real es Polars/Python. Si la estimación requiere PySpark, el plan rechaza con ENGINE_UNAVAILABLE; disco insuficiente produce RESOURCE_DISK_INSUFFICIENT. No hay fallback silencioso ni adapter Spark instalado. DuckDB se usa en metadata y paridad de expresiones, no como pipeline completo seleccionable.

La presencia de timeout_seconds en el plan no equivale a un límite duro por proceso. La interrupción bajo carga, kill/restart de procesos y el presupuesto real de memoria requieren certificación de estrés adicional. La cancelación es cooperativa; Redis/Celery, prioridades/lanes y workers distribuidos son futuros.

17. Modelo persistente y migraciones

PostgreSQL mantiene 17 tablas de aplicación. El esquema real es más compacto que el ERD objetivo inicial: organización se expresa mediante organization_id lógico y roles mediante users.role más política central. No existen aún tablas dedicadas organizations/roles/permissions.

Tabla	Identidad / relación principal
users	Usuario, organization_id, role, password_hash, active.
sessions	Token hash, usuario, CSRF y expiración.
datasets	Nombre único por organización, área, owner, criticidad y estado.
dataset_versions	Versión única por dataset, schema/profile, artifacts y lineage.
configurations	Snapshot por módulo; previous_version_id y datasets.
runs	Snapshot de inputs/config, actor, plan, métricas, decisión y evidencia.
jobs	Run único, estado, intentos y lease.
findings	Run, código, fingerprint único por Run y detalles.
exceptions	Finding único, configuración, origen, validación y timeline/version.
audit_events	Actor estable, sujeto, request/run y metadata sanitizada.
idempotency_keys	Organización/ruta/key, hash request y Run.
artifacts	Identidad inmutable, kind, locator, hash/tamaño/MIME.
artifact_links	Relación dirigida única entre entidades.
metric_history	Métrica/dimensión/run, método/version y valor.
external_connections	Identidad, revisión vigente, estado y última prueba; ámbito por organización.
external_connection_versions	Configuración inmutable, referencia de secreto y config_hash.
dataset_source_bindings	Dataset, conexión, schema, tabla/vista y overrides.

Cadena de trazabilidad

Dataset -> DatasetVersion -> Configuration -> Run -> Finding -> Exception. La excepción añade Validation Run de la misma Configuration. ArtifactLink conecta originales, canónicos, outputs, manifests y exports. Las claves de organización y comprobaciones del servicio acotan todos los accesos.

Migración	Cambio
0001_initial	Esquema original.
0002_evidence_v2	Identidad, artefactos, linaje, evidencia y métricas versionadas.
0003_dataset_ingestion_metadata	Formato/lector/opciones/esquema nativo por versión.
0004_exception_validation	Configuración de origen, validación técnica y cierre administrativo.
0005_external_connections	Conexiones externas, configuraciones versionadas y bindings de datasets.

La incorporación de Conexiones usa una migración nueva aditiva. API startup aplica Alembic y backfill idempotente. SQLite legacy sólo se adopta cuando coincide con un schema reconocido; no se fuerza sobre estructuras desconocidas. Downgrade/upgrade se prueban en bases temporales, nunca sobre datos operativos.

18. API y errores

La base es /api/v1. OpenAPI generado desde FastAPI y backend/API_CONTRACT.md son las referencias de campos exactos. El inventario siguiente se genera desde backend/openapi.json y conserva rutas reales, incluidos alias y compatibilidad deprecated.

Método	Ruta	Operación
GET	/health	Health
GET	/health/ready	Ready
GET	/api/v1/health	Health
GET	/api/v1/health/ready	Ready
POST	/api/v1/auth/demo	Demo Login
POST	/api/v1/auth/login	Login
GET	/api/v1/me	Me
POST	/api/v1/auth/logout	Logout
GET	/api/v1/datasets	Datasets
POST	/api/v1/datasets	New Dataset
POST	/api/v1/datasets/uploads/inspect	Inspect Dataset Upload
GET	/api/v1/datasets/{dataset_id}	Dataset Detail
GET	/api/v1/datasets/{dataset_id}/schema	Dataset Schema
POST	/api/v1/datasets/{dataset_id}/versions/upload	Upload Version
GET	/api/v1/dataset-versions/{version_id}/profile	Profile
GET	/api/v1/monitors	Configurations
POST	/api/v1/monitors	Monitor
GET	/api/v1/recon/controls	Configurations
POST	/api/v1/recon/controls	Recon Control
GET	/api/v1/intake/contracts	Configurations
POST	/api/v1/intake/contracts	Intake Contract
POST	/api/v1/intake/runs	Intake Run
POST	/api/v1/recon/runs	Recon Run
POST	/api/v1/monitors/{monitor_id}/runs	Monitor Run
GET	/api/v1/runs	Runs
GET	/api/v1/runs/{run_id}	Run Detail
POST	/api/v1/runs/{run_id}/cancel	Cancel Run
GET	/api/v1/recon/runs/{run_id}/results	Results
GET	/api/v1/intake/runs/{run_id}/errors	Results
GET	/api/v1/runs/{run_id}/results	Results
GET	/api/v1/runs/{run_id}/evidence	Evidence
GET	/api/v1/artifacts/{artifact_id}/download	Artifact Download
GET	/api/v1/runs/{run_id}/export.xlsx	Export Excel

Método	Ruta	Operación
GET	/api/v1/runs/{run_id}/export.csv	Export Csv (legacy)
GET	/api/v1/monitors/{monitor_id}/metrics	Monitor Metrics
GET	/api/v1/findings	Findings
POST	/api/v1/findings/{finding_id}/exceptions	Finding Exception
GET	/api/v1/exceptions	Exceptions
GET	/api/v1/exceptions/{case_id}	Exception Detail
PATCH	/api/v1/exceptions/{case_id}	Update Exception
POST	/api/v1/exceptions/{case_id}/validate	Validate Exception
GET	/api/v1/audit-events	Audit Events
GET	/api/v1/rules	Rules
GET	/api/v1/users	Users
GET	/api/v1/system/engines	Engines
GET	/api/v1/dashboard	Dashboard
POST	/api/v1/monitors/{configuration_id}/versions	Publish Configuration Version
POST	/api/v1/recon/controls/{configuration_id}/versions	Publish Configuration Version
POST	/api/v1/intake/contracts/{configuration_id}/versions	Publish Configuration Version
GET	/api/v1/runs/{run_id}/execution-plan	Run Execution Plan
GET	/api/v1/runs/{run_id}/diagnostics	Run Diagnostics
POST	/api/v1/execution-plans/preview	Preview Execution
GET	/api/v1/connections	List Connections
POST	/api/v1/connections	Create Connection
POST	/api/v1/connections/test	Test Draft
GET	/api/v1/connections/{connection_id}	Connection Detail
PATCH	/api/v1/connections/{connection_id}	Update Connection
DELETE	/api/v1/connections/{connection_id}	Delete Connection
POST	/api/v1/connections/{connection_id}/test	Test Connection
GET	/api/v1/connections/{connection_id}/schemas	Schemas
GET	/api/v1/connections/{connection_id}/objects	Objects
GET	/api/v1/connections/{connection_id}/preview	Preview
POST	/api/v1/connections/{connection_id}/datasets	Register Source Dataset
POST	/api/v1/datasets/{dataset_id}/refresh-source	Refresh Source

19. Contratos HTTP y permisos

Las listas comunes devuelven {items,total}; IDs son strings opacos y fechas ISO UTC. El cockpit devuelve un DTO agregado. Los downloads usan el MIME del artefacto verificado y filename sanitizado.

Autenticación utiliza cookie HttpOnly trackvance_session. Mutaciones autenticadas requieren X-CSRF-Token y validación de origen. Sesiones duran 12 horas en el prototipo. Passwords se verifican mediante Argon2. El acceso demo es explícito, depende de DEMO_ACCESS_ENABLED y no representa SSO de producción. DEMO_SEED_ENABLED controla por separado la creación de datos sintéticos.

Rol	Alcance local
Administrator	Lectura, autoría, ejecución, exports, auditoría, sistema, cierre de excepciones y administración/uso de conexiones.
Data Owner / Lead	Lectura/autoría/ejecución, exports, auditoría, cierre y administración/uso de conexiones.
Data Analyst	Lectura/autoría/ejecución, exports, auditoría y uso de conexiones; sin administración de conexiones ni cierre de excepciones.
Operations	Lectura/exports y gestión de excepciones según permiso; solo consulta metadata de conexiones.
Auditor	Lectura/exports/auditoría/usuarios/sistema y metadata de conexiones; sin mutaciones operativas.

permissions.py es la política central; roles desconocidos no reciben permisos. Se distinguen datasets:read/write, configurations:write, runs:read/execute, exceptions:read/write/close, connections:read/use/manage, exports:download, artifacts:download, audit:read, users:read/write y system:read. connections:read permite consultar metadata; use permite probar una conexión guardada, explorar y adquirir snapshots; manage permite crear, editar, deshabilitar, eliminar y probar borradores. Los cierres técnicos y administrativos requieren exceptions:close además del permiso de gestión. No existe todavía un CRUD completo de usuarios/roles/organizaciones pese a que algunos permisos preparen esa evolución.

Errores operativos

El envelope es {error:{code,message,details,request_id}}. Sesión inválida devuelve 401; permiso/CSRF 403; recurso no accesible 404; conflicto/integridad 409; validación 422; readiness fallido 503. La UI conserva el mensaje legible y la referencia de diagnóstico.

VALIDATION_RUN_REQUIRED, VALIDATION_RUN_OUTDATED, TECHNICAL_VALIDATION_REQUIRED y ADMINISTRATIVE_REASON_REQUIRED protegen las excepciones. VERSION_CONFLICT evita sobrescritura por formularios obsoletos. UNSUPPORTED_FORMAT, INVALID_DATA y UPLOAD_TOO_LARGE explican rechazos de carga. DATASET_NAME_EXISTS distingue el nombre duplicado de un conflicto genérico.

Hardening objetivo: rate limiting, administración avanzada de sesiones, OIDC, sensibilidad/masking, política de retención, autorización por asignación y pruebas de penetración. No deben asumirse por existir sesión/RBAC local.

20. Configuración, operación y respaldo

La configuración es por entorno y defaults locales. `.env.example` es una plantilla sin secretos operativos. `.env`, `storage`, datos de usuario, `caches`, `logs`, resultados temporales y backups se excluyen de Git y del contexto Docker según `.gitignore/.dockerignore`.

Variable	Función / default
DATABASE_URL	PostgreSQL en Compose; SQLite local en ejecución directa.
TRACKVANCE_STORAGE_DIR / ROOT	Raíz interna; <code>/var/lib/trackvance</code> en Compose.
TRACKVANCE_WEB_ORIGIN	Origen esperado; debe coincidir con <code>WEB_PORT</code> .
COMPOSE_PROJECT_NAME / WEB_PORT	Instalación y puerto; no confundir volúmenes entre proyectos.
POSTGRES_DB/USER/PASSWORD	Acceso local a metadata; password sólo en entorno.
TRACKVANCE_SECRETS_DIR	Volumen de credenciales cifradas externas.
TRACKVANCE_SECRET_KEY_FILE	Clave maestra local en volumen independiente.
DEMO_ACCESS_ENABLED	Habilita únicamente el login demo explícito; garantiza la identidad y organización mínimas, no datos sintéticos.
DEMO_SEED_ENABLED	Habilita únicamente el seed idempotente de datasets, configuraciones, ejecuciones, hallazgos y excepciones demo.
MAX_UPLOAD_BYTES / TRACKVANCE_MAX_ROWS	10 MiB y 100.000 filas por defecto.
TRACKVANCE_WORKER_MEMORY_SOFT_BYTES	Presupuesto blando del planner.
TRACKVANCE_TEMP_MIN_FREE_BYTES	Reserva mínima de disco temporal.
TRACKVANCE_RUN_TIMEOUT_SECONDS	Valor del plan; no timeout duro certificado.

Para conservar el cierre seguro de instalaciones anteriores, si `DEMO_ACCESS_ENABLED` está ausente hereda el valor de `DEMO_SEED_ENABLED`. Al definir ambas variables explícitamente sus comportamientos son independientes.

`TRACKVANCE_ENV` aparece como etiqueta en la plantilla; no selecciona por sí sola una política de seguridad productiva. No hay Settings Pydantic ni feature flags persistidos actualmente.

Readiness comprueba SQL, Alembic head y escritura temporal. `doctor.py` comprueba HTTP, servicios y heartbeat. `verify_storage.py` snapshot calcula hashes de registros de negocio y valida todos los artifacts; compare exige igualdad antes/después de un reinicio en reposo. Entre capturas no deben ejecutarse login/exports/tests que agregan auditoría legítima.

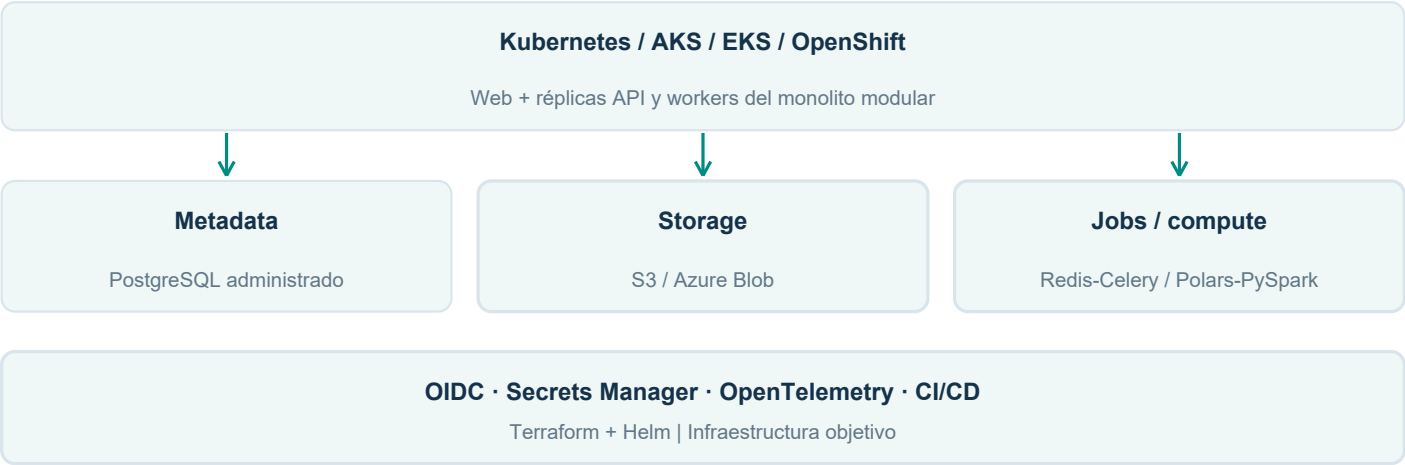
El respaldo SQLite usa una copia consistente, integra WAL, verifica hashes y restaura en destino nuevo con rutas reubicadas. Incluye credenciales cifradas y clave maestra; una referencia de conexión sin su secreto o clave invalida el respaldo. La restauración conserva estos archivos en `credentials/` y `keys/` del destino con acceso restringido y mantiene compatibilidad con backups históricos sin conexiones.

PostgreSQL requiere pausar escrituras, `pg_dump` y copia coordinada de `trackvance_data`, `connection_credentials` y `connection_keys`, conservando hashes y permisos. Se restaura el conjunto en una base y volúmenes nuevos; la clave debe corresponder a las credenciales cifradas y permanece separada de ellas. Se verifican Alembic, readiness, artifacts y una conexión autorizada antes de usar el destino. El backup integral automático PostgreSQL+artifacts+secretos sigue pendiente; no se sustituye con una copia de archivos del data directory en caliente.

Para actualizar: terminar runs, respaldar, reconstruir imágenes, aplicar Alembic al iniciar, esperar healthchecks y comparar almacenamiento. Nunca ejecutar `down -v` sobre la instalación del usuario como parte de un upgrade.

21. Arquitectura final de producto

OBJETIVO. El producto final conserva los mismos módulos y contratos funcionales. Evoluciona la entrega de jobs, almacenamiento, fuentes, identidad, observabilidad e infraestructura. No divide Intake, ReconOps o Sentinel en microservicios.



Capacidad	Adaptador / infraestructura objetivo
Despliegue	Docker y Kubernetes, AKS, EKS u OpenShift; Helm para releases.
Metadata	PostgreSQL administrado, backups y recuperación verificados.
Storage interno	S3 o Azure Blob mediante StorageProvider, locators, hashes y retención.
Fuentes	PostgreSQL y SQL Server implementados; S3, Azure Blob, APIs y otros motores futuros mediante DatasetSource.
Jobs	Redis + Celery, reintentos/leases e idempotencia; entrega transaccional definida.
Compute	Polars para cargas estándar, PySpark para grandes volúmenes con paridad certificada.
Identidad	OAuth2/OIDC + SSO con Entra ID, Keycloak u otro IdP; RBAC/org scope propios.
Secretos	Key Vault, AWS Secrets Manager o Vault; referencias fuera de configs de negocio.
Observabilidad	OpenTelemetry, Prometheus, Grafana y logs centralizados.
Infraestructura	Terraform y configuración por ambiente, redes y permisos mínimos.
CI/CD	Quality gates, análisis de dependencias/SAST/secretos/imágenes, SBOM y despliegue controlado.

Pasos de evolución sin cambiar reglas

1. Introducir locator remoto y materialización acotada en un StorageProvider, migrando referencias y validando hashes históricos.
2. Ampliar PostgreSQL/SQL Server con otros conectores DatasetSource, límites, credenciales externas y snapshots reproducibles.
3. Implementar JobQueue Redis/Celery con patrón transaccional/outbox o garantía equivalente; conservar Run y evidencia idempotentes.
4. Añadir ExecutionEngine PySpark, capacidades del planner y pruebas de paridad antes de habilitarlo.
5. Integrar IdP, secretos y observabilidad; aplicar tests de aislamiento y recuperación.
6. Desplegar réplicas API/workers y certificar concurrencia, presión de recursos, backup/restore y upgrades.

Los puertos actuales reducen el área de cambio, pero no certifican esos adapters. La transición requerirá código de infraestructura y migraciones; no deberá reescribir la semántica de reglas, resultados ni trazabilidad.

22. Calidad, pruebas y aceptación

La certificación local ejecuta pytest, Ruff, Mypy, ESLint, TypeScript, Vitest, build frontend, Playwright, smoke API, migraciones PostgreSQL y prueba de persistencia tras restart. El runner crea un proyecto Compose temporal con sus propios volúmenes y elimina únicamente esos datos al terminar.

Verificación	Resultado ejecutado
pytest backend + operaciones	363 aprobadas; API, seguridad, adapters, backups y runner de Conexiones. Dos avisos de deprecación de dependencias.
Ruff / Mypy	Aprobados; Mypy revisó 27 archivos fuente.
ESLint / TypeScript / build	Aprobados. Build Vite: aviso de bundle de 500,53 kB (151,44 kB gzip); sin error de compilación.
Vitest	89 pruebas aprobadas en 9 archivos.
Playwright	15 flujos aprobados en Docker con ambos motores (48,2 s); el escenario opt-in de acceso limpio pasó por separado (1,1 s). Total: 16.
Smoke API / doctor	84 comprobaciones aprobadas; doctor 7/7.
Conexiones reales	PostgreSQL y SQL Server: 62 comprobaciones de conexión, permisos, metadata, snapshot, Intake, caída y secretos aprobadas.
Migraciones PostgreSQL	Upgrade histórico hasta 0005, paridad ORM, Alembic check y downgrade/upgrade en base temporal: aprobados.
Restart aislado	Registros y hashes idénticos: 45 datasets, 59 versiones, 37 runs y 173 artifacts verificados.
Instancia principal	Cuatro servicios healthy en localhost:3100, migración 0005, restart=no. Sin datasets, versiones, runs ni conexiones de prueba; acceso demo sin seed.
Limpieza de pruebas	Proyectos E2E y volúmenes temporales eliminados. Datos de usuario preservados.
Backup SQLite y secretos	Backup v2 y restore conservan credenciales utilizables; hashes, corrupción, referencias, traversal y compatibilidad v1 verificados.
Seguridad de conexión	Cambio de destino/TLS exige contraseña; cierre de excepción exige permiso; DELETE exige versión vigente; solo API monta secretos y claves.

Escenarios de negocio conservados

Escenario	Resultado certificado por tests
Intake 120 filas	110 válidas, 10 con error, aceptación 91,67 %, REJECTED.
Detalle Intake	REQUIRED 4, UNIQUE 4 registros, POSITIVE 2.
Recon 120/110	MATCH 110, SOURCE_ONLY 6, DUPLICATE_SOURCE 4; otras categorías 0; match 91,67 %.
Sentinel baseline	110 filas, 9 checks, 0 fallos, salud 100 %.
Sentinel v2	90 filas, 3 nulls monitorizados, cambio 18,18 %, 4 fallos, salud 55,56 %.
Sentinel v3	90 filas, source_system ausente, cambio 0 %, 1 fallo, salud 88,89 %.
Excepciones	Bloqueo sin validación; Run posterior correcto habilita RESOLVED; cierre administrativo separado.

La cobertura nueva incluye cinco formatos, hoja Excel, delimitador TXT, JSON anidado simple, esquema embebido Parquet, overrides, identificadores/Todos, selectores de reglas, navegación, dashboard, VALUE_MISMATCH, not_future, schema drift, XLSX/MIME/filename/formula injection, RBAC/CSRF y lineage.

El workflow ci.yml contiene jobs backend, frontend, compose-e2e y connections-e2e, con migraciones en PostgreSQL temporal y evidencia de navegador. Conexiones se prueba con motores reales PostgreSQL y SQL Server, usuarios SELECT, caída/reconexión, versiones, Intake y búsqueda de secretos en logs/metadata. La ejecución local no equivale a una ejecución remota de GitHub Actions. Security scanning/SBOM y gates de producción permanecen en el objetivo final.

Pendientes explícitos

PySpark y conectores adicionales; adquisición de gran volumen por jobs; Data Delivery; rotación automática de clave maestra; Redis/Celery; OIDC/SSO; Kubernetes/Helm/Terraform; observabilidad distribuida; stress/timeout duro; backup integral PostgreSQL automatizado; masking/sensibilidad; agrupación multi-finding y resolución automática. Estos pendientes no bloquean la operación local acotada y no se presentan como capacidades ya instaladas.

23. Decisiones vigentes y control documental

Decisión	Referencia del repositorio
Monolito modular local y dos modos de arranque	docs/adr/0001-local-prototype.md, antecedente 0.1.
Evidencia v2, actores, artefactos y compatibilidad	docs/adr/0002-evidence-and-artifacts.md.
Semántica observada y reglas declarativas	docs/adr/0003-rule-semantics.md.
Lectores y frontera común de calidad	docs/adr/0004-dataset-readers.md.
Resolución de excepciones validada técnicamente	docs/adr/0005-technical-validation-of-exceptions.md.
Puertos de storage, fuentes, ejecución y cola	docs/adr/0006-architecture-ports.md.
Conexiones, secretos, snapshots y frontera Data Delivery	docs/adr/0007-external-connections.md.

Precedencia y cambios de esta revisión

Esta revisión sustituye las afirmaciones de implementación de la edición inicial de 52 páginas. La edición inicial se conserva archivada con su hash, como antecedente. Los diagramas aspiracionales de worker-spark, tablas de organizaciones/roles, librerías no instaladas, exports CSV principales y endpoints futuros ya no se presentan como realidad operativa.

Se actualizan la arquitectura local/final, esquema real de 17 tablas, API generada, cinco lectores, Conexiones PostgreSQL/SQL Server, SecretStore, snapshots, profiling exacto, overrides, catálogo de reglas, key normalization, Parquet Intake, manifests v2, Excel, cockpit, RBAC, auditoría, excepciones técnicas, persistencia Docker y arranque manual.

El código y OpenAPI determinan el contrato ejecutable. Un cambio posterior de semántica requiere ADR, pruebas, actualización de esta fuente y del PDF; un cambio de base de datos requiere Alembic nuevo. Las versiones de configs/manifests/métricas se gestionan separadas de la versión del producto.

Registro

Fecha	Revisión	Descripción
10-09-2026	Especificación v1.1 inicial	Diseño y objetivo de producto.
16-09-2026	v1.1, implementación 0.3.0	Consolidación del estado real, fronteras de infraestructura y validación local.
19-09-2026	v1.1, evolución Conexiones	PostgreSQL, SQL Server, secretos cifrados, snapshots y pruebas con motores reales.

La fuente editable acompaña al PDF en Documentación. README, docs/architecture.md, catálogo, ADRs, OpenAPI y documentación de operación complementan esta especificación con comandos y contratos de detalle.